UNIT III: Constructors, Destructors and Operator Overloading

Answer Key: Debugging and Pseudo-Code Formulation

Name: Kavitha B **Registration No:** 192565041

How to read this document. Section A answers the five debugging problems in the format required by the question bank: line numbers and bug classification, root-cause analysis, corrected compilable code, and the dry-run output. Section B gives the five pseudo-code designs with dry-run trace tables.

Verification. Every corrected C++ program in Section A was compiled with g++ (-std=c++17 -Wall -Wextra) and run; the console outputs shown are the real outputs. The compiler messages quoted for the buggy code are the real g++ messages, and the buggy programs for Q2 and Q4 were also run to observe the faulty behaviour.

Line numbers. Lines are quoted as labelled by the // Line N comments in the question paper. g++ counts physical lines of the pasted code and can differ by a few lines; where it matters, the g++ line is given in brackets.

Section A: Debugging Questions

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

Defect classification

Line	Defect	Classification
17 (g++ line 18)	Copy constructor DynamicString(DynamicString source) takes its parameter by value	Compilation error: invalid copy constructor signature (ill-formed declaration)

Task 1: The exact compiler error

g++ reports: error: invalid constructor; you probably meant 'DynamicString (const DynamicString&)'. Clang and MSVC reject the declaration as well, each with their own wording (both say that a copy constructor must take its first parameter by reference).

Task 2: Why by-value leads to infinite recursion

Passing an argument by value means the parameter is initialised as a copy of the argument. For a class type, that copy is made by the class's copy constructor. So for DynamicString s2 = s1; to run DynamicString(DynamicString source), the compiler would first have to build the parameter source from s1, which needs a call to the copy constructor, which again needs a parameter built from s1, and so on forever. The regress never reaches a base case.

The C++ standard therefore makes such a declaration ill-formed ([class.copy.ctor]: a constructor whose first parameter is of type X, with no other required parameters, is invalid). The compiler detects the impossibility while compiling and stops, instead of producing a program that would recurse until the stack overflowed.

Task 3: The corrected signature and the role of const and &

```
DynamicString(const DynamicString &source)
```

- **& (reference):** the parameter becomes an alias for the caller's object, so no copy is made and the recursion disappears. It also avoids the cost of copying the object.

- **const:** it promises that the constructor will not modify the source. This lets the constructor accept const objects and temporaries (a non-const reference cannot bind to them), and it prevents accidental changes to the original while copying.

Task 4: Complete corrected code

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char *buffer;
    int length;

public:
    // Parameterized Constructor
    DynamicString(const char *str) {
        length = std::strlen(str);
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    // Copy Constructor: parameter is a const reference
    DynamicString(const DynamicString &source) {
        length = source.length;
        buffer = new char[length + 1];          // fresh heap block (deep copy)
        std::strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << std::endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;   // copy construction
    s2.print();
    return 0;
}
```

Dry run:

Step	Statement	State
1	DynamicString s1("Antigravity C++");	strlen = 15; heap block H1 of 16 bytes; buffer = "Antigravity C++"
2	DynamicString s2 = s1;	copy constructor runs: length = 15; a new block H2 of 16 bytes is allocated; strcpy copies the text. s1 and s2 now own different blocks (deep copy)
3	s2.print();	prints the text held in H2
4	end of main()	destructors run in reverse order: s2 frees H2, then s1 frees H1 (each block freed exactly once)

Console output

```
Antigravity C++
```

Question 2: Shallow Copy & Double-Free Runtime Crash

Defect classification

Location	Defect	Classification
Class AudioBuffer (the comment "Programmer omitted copy constructor!")	No user-defined copy constructor although the class owns heap memory (Rule of Three violated)	Runtime error: shallow copy, pointer aliasing
Line 24 / destructor (delete[] samples;)	The same block is deleted twice, once for the copy and once for the original	Runtime error: double free, heap corruption (SIGABRT)
processAudio(AudioBuffer buf)	Object passed by value triggers the copy	Exposes the defect

Running the buggy program produced the following, which confirms the analysis:

Console output

```
Buffer allocated at: 0x556081bc52b0
Processing sample 0: 0
Deallocating buffer at: 0x556081bc52b0
Back in main, sample 0: 1443371973
Deallocating buffer at: 0x556081bc52b0
free(): double free detected in tcache 2
```

Task 1: Address trace and the synthesized copy constructor

Moment	Address held in samples	Remark
Before processAudio(track)	track.samples = A (0x...52b0)	one allocation, made by the constructor
During processAudio	buf.samples = A (the same address)	buf is a member-wise copy of track; no new allocation, no "Buffer allocated" message
After processAudio returns	buf destroyed, so delete[] A runs; track.samples still holds A	track.samples is now a dangling pointer (the block has been freed)
Back in main()	reads track.samples[0] through the dangling pointer	use after free: undefined behavior, the garbage value 1443371973 was printed
End of main()	track destroyed, so delete[] A runs a second time	double free: the program aborts

The compiler-synthesized copy constructor copies each data member one by one ([class.copy.ctor], implicitly-defined copy constructor). For a raw pointer member this copies only the address value, not the data it points to. The result is a shallow copy: two objects whose pointers alias one heap block.

Task 2: Why free(): double free detected appears

Both objects believe they own block A, and both destructors call delete[] samples. After the first delete[] the allocator puts the block on its free list (glibc's tcache). The second delete[] hands the allocator a block that is already free. Deleting memory that was already deallocated is undefined behavior ([expr.delete]), and glibc's integrity check catches it, prints free(): double free detected and aborts the process. The root cause is the broken ownership: one allocation, two owners.

Task 3: Deep copy constructor

```
AudioBuffer(const AudioBuffer &other) : capacity(other.capacity) {
    samples = new int[capacity];                // fresh memory
    for (int i = 0; i < capacity; ++i) {
        samples[i] = other.samples[i];          // element-by-element copy
```

```
    }
    std::cout << "Deep copy allocated at: " << samples << std::endl;
}
```

Note: the Rule of Three also calls for a user-defined copy assignment operator whenever a destructor and a copy constructor are defined; it is not needed in this program, but any code doing a = b would still share memory.

Task 4: Complete corrected program, dry run and verified output

```
#include <iostream>

class AudioBuffer {
public:
    int *samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) {
            samples[i] = i * 10;
        }
        std::cout << "Buffer allocated at: " << samples << std::endl;
    }

    // Deep copy constructor
    AudioBuffer(const AudioBuffer &other) : capacity(other.capacity) {
        samples = new int[capacity];                // fresh memory
        for (int i = 0; i < capacity; ++i) {
            samples[i] = other.samples[i];          // element-by-element copy
        }
        std::cout << "Deep copy allocated at: " << samples << std::endl;
    }

    ~AudioBuffer() {
        std::cout << "Deallocating buffer at: " << samples << std::endl;
        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {                // still passed by value
    std::cout << "Processing sample 0: " << buf.samples[0] << std::endl;
}

int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: " << track.samples[0] << std::endl;
    return 0;
}
```

Step	Action	Heap state	Output
1	AudioBuffer track(5)	block A (5 ints: 0, 10, 20, 30, 40)	Buffer allocated at: A
2	processAudio(track): copy constructor builds buf	block A and a new block B, with B[i] = A[i]	Deep copy allocated at: B
3	buf.samples[0] printed	A, B	Processing sample 0: 0
4	processAudio ends: buf destroyed	B freed, A still valid	Deallocating buffer at: B
5	main prints track.samples[0]	A valid	Back in main, sample 0: 0
6	end of main: track destroyed	A freed, heap empty	Deallocating buffer at: A

Console output

```
Buffer allocated at: 0x55f8d29172b0
Deep copy allocated at: 0x55f8d29182e0
Processing sample 0: 0
Deallocating buffer at: 0x55f8d29182e0
Back in main, sample 0: 0
Deallocating buffer at: 0x55f8d29172b0
```

The addresses change from run to run; what matters is that there are two distinct allocations, each freed exactly once, and that main() still reads valid data.

Question 3: Stream Insertion Operator << Member Function Violation

Defect classification

Line	Defect	Classification
11	operator<< declared as a member of Fraction	Design / grammar error: wrong operand order (the object becomes the left operand)
20 (g++ line 22)	std::cout << f1 << std::endl;	Compilation error: no match for 'operator<<' (operand types are 'std::ostream' and 'Fraction')
23	f1 << std::cout;	Compiles, but inverted semantics: unacceptable in production code

Task 1: How the compiler translates the operator call

For a binary operator expression a @ b the compiler looks for either a member function a.operator@(b) or a non-member function operator@(a, b) ([over.match.oper]). The member candidates are searched only in the class of the left operand.

In std::cout << f1 the left operand is a std::ostream, so the member candidates are the operator<< members of std::ostream (for int, double, const char*, bool and so on). Fraction::operator<< is a member of Fraction and is considered only when the left operand is a Fraction. None of the ostream candidates can accept a Fraction, and there is no non-member operator<<(std::ostream&, const Fraction&). Since we cannot add a member to the standard library class, no function matches and compilation fails.

Task 2: Why f1 << std::cout compiles, and why it is unacceptable

f1 << std::cout is translated to f1.operator<<(std::cout), which matches the member exactly, so it compiles. It is unacceptable because it reverses the universal convention that the stream is on the left: it cannot be mixed in the usual output chain (std::cout << "Value: " << f1 does not work), it confuses readers (it looks like a bit-shift), and it breaks every library and code convention for stream output.

Task 3: Friend non-member operator<<

```
friend std::ostream& operator<<(std::ostream &os, const Fraction &f) {
    os << f.numerator << "/" << f.denominator;
    return os;
}
```

- **Non-member:** the stream must be the left operand, so the function cannot belong to Fraction.
- **friend:** it needs access to the private numerator and denominator.
- **std::ostream& parameter:** the stream is modified by writing to it, and it must not be copied (streams are non-copyable).
- **const Fraction&:** no copy of the fraction is made and the function cannot modify it.

- **Returns std::ostream&:** this makes chaining possible: (std::cout << a) << b.

Task 4: Corrected code, dry run and output

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    // Non-member friend: left operand is the stream, right operand is the Fraction
    friend std::ostream& operator<<(std::ostream &os, const Fraction &f) {
        os << f.numerator << "/" << f.denominator;
        return os;                      // returned to allow chaining
    }
};

int main() {
    Fraction f1(3, 4);
    Fraction f2(5, 8);

    std::cout << f1 << std::endl;
    std::cout << "Fractions: " << f1 << " and " << f2 << std::endl;   // chaining
    return 0;
}
```

Step	Expression	Resolution / result
1	std::cout << f1	calls operator<<(std::cout, f1): writes 3/4 and returns cout
2	... << std::endl	applies to the returned cout: newline and flush
3	std::cout << "Fractions: " << f1 << " and " << f2	each << returns the stream, so the chain prints Fractions: 3/4 and 5/8

Console output

```
3/4
Fractions: 3/4 and 5/8
```

Question 4: Prefix vs. Postfix Increment Signature Inversion

Defect classification

Line	Defect	Classification
10	StepCounter operator++() has no int parameter, so it is the PREFIX operator, yet it saves a temp and returns the old value by value	Logic error: prefix has postfix semantics
17	StepCounter& operator++(int) has the dummy int, so it is the POSTFIX operator, yet it increments and returns a reference to the new value	Logic error: postfix has prefix semantics
main(): StepCounter c2 = ++c1;	calls the zero-parameter operator, which returns the old value 10	Symptom: c2 shows 10 instead of 11

The buggy program was run and printed Steps: 10 for c2 and Steps: 11 for c1, exactly the symptom described in the question.

Task 1: The standard signatures

Operator	Member signature	Non-member form
Prefix ++obj	T& operator++();	T& operator++(T&);
Postfix obj++	T operator++(int);	T operator++(T&, int);

Task 2: Purpose of the dummy int parameter

Both operators are written with the same name, operator++, and the call syntax ++x and x++ is the only difference. Functions cannot be overloaded on return type, so the language needs a difference in the parameter list. By convention ([over.inc]) the postfix form has an extra unnamed int parameter. It carries no data: when the compiler translates x++ into x.operator++(int), it passes the value 0, and the body never uses it. Its only job is to give the postfix function a different signature from the prefix one.

Task 3: Why prefix returns by reference, postfix by value

- **Prefix:** the object is modified first, and the expression ++x means the modified object itself. Returning *this by reference avoids a copy, matches the built-in type (++x is an lvalue, so ++(++x) works), and is safe because *this outlives the call.
- **Postfix:** the expression x++ must yield the OLD value while x itself changes. The old state exists only as a local temporary inside the function. Returning a reference to it would be a dangling reference (Unit II, Debugging Q1), so the function returns the temporary by value, a copy. This extra copy is why ++x is preferred over x++ for class types when the old value is not needed.

Task 4: Corrected code, dry run and output

```
#include <iostream>

class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    // PREFIX: no parameter, increment first, return reference to the modified object
    StepCounter& operator++() {
        ++steps;
        return *this;
    }

    // POSTFIX: dummy int, save old state, increment, return the old copy by value
    StepCounter operator++(int) {
        StepCounter temp = *this;
        ++steps;
        return temp;
    }

    void display() const {
        std::cout << "Steps: " << steps << std::endl;
    }
};

int main() {
    StepCounter c1(10);

    StepCounter c2 = ++c1;      // prefix: c1 becomes 11, c2 gets 11
```

```
    std::cout << "After c2 = ++c1:" << std::endl;
    c2.display();
    c1.display();

    StepCounter c3 = c1++;      // postfix: c3 gets old 11, c1 becomes 12
    std::cout << "After c3 = c1++:" << std::endl;
    c3.display();
    c1.display();

    ++(++c1);                   // prefix chaining works because of the reference return
    std::cout << "After ++(++c1):" << std::endl;
    c1.display();
    return 0;
}
```

Step	Statement	c1.steps	Other object
1	StepCounter c1(10)	10	-
2	c2 = ++c1 (prefix: increment, return reference)	11	c2 = copy of c1 = 11
3	c3 = c1++ (postfix: temp = 11, increment, return temp)	12	c3 = 11 (old value)
4	++(++c1): inner call returns a reference to c1, the outer call increments the same object	13, then 14	-

Console output

```
After c2 = ++c1:
Steps: 11
Steps: 11
After c3 = c1++:
Steps: 11
Steps: 12
After ++(++c1):
Steps: 14
```

Question 5: Violation of Fundamental Operator Overloading Rules

Defect classification

Line	Defect	Classification
8 (g++ line 9)	bool operator.(...): attempting to overload the member-access operator	Compilation error (grammar): expected type-specifier before '.' token
14 (g++ line 15)	int operator+(int, int): operator for built-in types only	Compilation error: 'int operator+(int, int)' must have an argument of class or enumerated type
19 (g++ line 20)	MathVector operator**(MathVector, int): inventing a new operator	Compilation error (grammar): expected initializer before '*' token. Also, the body "// ..." returns nothing from a non-void function, which would be undefined behavior if it ever ran

Task 1: Operators that cannot be overloaded; why operator. is prohibited

The operators that can never be overloaded are: the scope resolution operator ::, the member access operator ., the pointer-to-member operator .*, the conditional operator ?: and sizeof. (typeid, alignof and the preprocessor symbols # and ## cannot be overloaded either.)

The dot operator's right operand is the name of a member, not a value or expression, so it cannot be passed to a function as an argument. More importantly, the dot is the basic way the language reaches the members of an object. If a class could redefine it, even the class's own code could no longer be sure what obj.member means, and ordinary member access would become unpredictable. (The arrow ->, by contrast, can be overloaded, which is how smart pointers work.) The same reasoning applies to :: and sizeof, whose operands are names or types.

Task 2: The rule behind the rejection of Line 14

An operator function must be either a member function or a non-member function with at least one parameter of class type or enumeration type (or a reference to one) ([over.oper]). int operator+(int, int) has only built-in parameter types, so it is rejected. This protects the meaning of expressions on built-in types: if it were allowed, the + in 2 + 3 would change meaning everywhere in the program (here, silently turning addition into multiplication).

Task 3: Can new operator tokens such as ** be defined?

No. Only the existing operator symbols can be overloaded; the set of tokens is fixed by the language grammar. The lexer reads ** as two separate * tokens, so operator** is parsed as operator* followed by a stray *, hence "expected initializer before '*'".

Overloading also changes none of the following: (1) precedence, so an overloaded * keeps the precedence of *; (2) associativity; (3) arity, so a unary operator stays unary and a binary operator stays binary, and an operator cannot be given a different number of operands. (Default arguments are not allowed on overloaded operators, except operator().) If ** were allowed, its precedence and arity would be undefined, which is exactly why the language forbids inventing operators.

Task 4: Cleanly rewritten class

```
#include <iostream>

class MathVector {
public:
    double x, y;
    MathVector(double a, double b) : x(a), y(b) {}

    // Legal: == (a different, overloadable operator, replaces operator.)
    bool operator==(const MathVector &other) const {
        return (x == other.x && y == other.y);
    }

    // Legal: + on a user-defined type (vector addition)
    MathVector operator+(const MathVector &other) const {
        return MathVector(x + other.x, y + other.y);
    }

    // Exponentiation as a named member method instead of the invented operator**
    MathVector power(int p) const {
        double rx = 1.0, ry = 1.0;
        for (int i = 0; i < p; ++i) { rx *= x; ry *= y; }
        return MathVector(rx, ry);       // component-wise power
    }
};

int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);

    std::cout << "v1 == v2 : " << (v1 == v2 ? "true" : "false") << std::endl;

    MathVector sum = v1 + v2;
    std::cout << "v1 + v2  : (" << sum.x << ", " << sum.y << ")" << std::endl;
```

```
    MathVector sq = v1.power(2);
    std::cout << "v1.power(2): (" << sq.x << ", " << sq.y << ")" << std::endl;

    std::cout << "int 2 + 3 : " << (2 + 3) << std::endl;   // built-in + unchanged
    return 0;
}
```

The invalid operator. is replaced by operator== (legal, and what the author actually wanted: equality of two vectors), the invalid int operator+ is replaced by operator+ on the class type MathVector, and the invented operator** is replaced by an ordinary named member method power(int). Here power() raises each coordinate to the given power; that meaning is a design choice.

Step	Statement	Result
1	v1(2.0, 3.0), v2(2.0, 3.0)	both vectors equal
2	v1 == v2	operator== compares x (2 = 2) and y (3 = 3): true
3	v1 + v2	operator+ returns (2 + 2, 3 + 3) = (4, 6)
4	v1.power(2)	x: 2 x 2 = 4; y: 3 x 3 = 9; returns (4, 9)
5	2 + 3 (int)	built-in addition is untouched: 5

Console output

```
v1 == v2 : true
v1 + v2  : (4, 6)
v1.power(2): (4, 9)
int 2 + 3 : 5
```

Section B: Pseudo-Code Writing Questions

Question 1: Dynamic String Engine with Deep Copy & Overloaded +

Task 1: Pseudo-code

```
CLASS DynStr
  PRIVATE:
    dataPtr   : POINTER TO CHARACTER
    charCount : INTEGER
  PUBLIC:
    CONSTRUCTOR(sourceString : STRING)
    COPY CONSTRUCTOR(REF other : DynStr)
    DESTRUCTOR()
    OPERATOR +(REF rhs : DynStr) RETURNS DynStr
    METHOD Display()
ENDCLASS

CONSTRUCTOR DynStr::DynStr(sourceString)
  charCount <- LENGTH(sourceString)
  dataPtr   <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER   // +1 for terminator
  FOR i <- 0 TO charCount - 1
    dataPtr[i] <- sourceString[i]
  NEXT i
  dataPtr[charCount] <- '\0'
END CONSTRUCTOR

COPY CONSTRUCTOR DynStr::DynStr(REF other)                  // DEEP copy
  charCount <- other.charCount
  dataPtr   <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER   // fresh buffer
  FOR i <- 0 TO charCount - 1
    dataPtr[i] <- other.dataPtr[i]                          // item-by-item
```

```
  NEXT i
  dataPtr[charCount] <- '\0'
END COPY CONSTRUCTOR

DESTRUCTOR DynStr::~DynStr()
  IF dataPtr <> NULL THEN
    DEALLOCATE dataPtr
    dataPtr <- NULL
  ENDIF
END DESTRUCTOR

OPERATOR DynStr::+(REF rhs) RETURNS DynStr
  total <- charCount + rhs.charCount
  temp  <- ALLOCATE ARRAY[total + 1] OF CHARACTER          // temporary buffer
  FOR i <- 0 TO charCount - 1                              // left operand text
    temp[i] <- dataPtr[i]
  NEXT i
  FOR j <- 0 TO rhs.charCount - 1                          // append right operand
    temp[charCount + j] <- rhs.dataPtr[j]
  NEXT j
  temp[total] <- '\0'
  result <- NEW DynStr(temp)                               // constructor allocates its own
  DEALLOCATE temp                                          // avoid a memory leak
  RETURN result                                            // by value
END OPERATOR

METHOD DynStr::Display()
  OUTPUT dataPtr
END METHOD

PROCEDURE Main
  strA <- DynStr("Hello ")
  strB <- DynStr("World!")
  strC <- strA + strB
  strC.Display()
END PROCEDURE                                              // destructors: strC, strB, strA
```

Task 2: Dry-run trace for strA = "Hello " and strB = "World!"

Step	Operation	Heap blocks live	Details
1	strA <- DynStr("Hello ")	A (7)	charCount = 6; ALLOCATE 7 chars; copy H, e, l, l, o, space; dataPtr[6] = terminator
2	strB <- DynStr("World!")	A (7), B (7)	charCount = 6; ALLOCATE 7 chars; copy W, o, r, l, d, !; terminator at index 6
3	strA + strB: total = 6 + 6 = 12; temp <- ALLOCATE 13	A, B, T (13)	copy 6 chars of strA to temp[0..5]; append 6 chars of strB to temp[6..11]; temp[12] = terminator; temp = "Hello World!"
4	result <- NEW DynStr(temp)	A, B, T, C (13)	charCount = 12; ALLOCATE 13 chars; copy 12 chars from temp; terminator
5	DEALLOCATE temp; RETURN result	A, B, C	T freed; result moves into strC (if the compiler does not elide the copy, the deep copy constructor allocates a new block and the temporary result is destroyed, the final state is the same)
6	strC.Display()	A, B, C	prints Hello World!
7	end of Main: destructors in reverse order	none	strC frees C, strB frees B, strA frees A; each dataPtr set to NULL; no leaks, no double free

Output: Hello World! Because the copy constructor allocates a fresh buffer, no two objects ever share a block (no shallow-copy aliasing).

Question 2: Complex Number System with Constructor Overloading & Operators

Task 1: Pseudo-code

```
CLASS Complex
  PRIVATE:
    real : REAL
    imag : REAL
  PUBLIC:
    CONSTRUCTOR()                              // 0 + 0i
    CONSTRUCTOR(r : REAL)                      // r + 0i
    CONSTRUCTOR(r : REAL, i : REAL)            // r + ii
    OPERATOR +(REF other : Complex) RETURNS Complex
    OPERATOR *(REF other : Complex) RETURNS Complex
    OPERATOR ==(REF other : Complex) RETURNS BOOLEAN
    METHOD Display()
ENDCLASS

CONSTRUCTOR Complex::Complex()
  real <- 0.0
  imag <- 0.0
END CONSTRUCTOR

CONSTRUCTOR Complex::Complex(r)
  real <- r
  imag <- 0.0
END CONSTRUCTOR

CONSTRUCTOR Complex::Complex(r, i)
  real <- r
  imag <- i
END CONSTRUCTOR

OPERATOR Complex::+(REF other) RETURNS Complex
  RETURN Complex(real + other.real, imag + other.imag)
END OPERATOR

OPERATOR Complex::*(REF other) RETURNS Complex
  // (a + bi)(c + di) = (ac - bd) + (ad + bc)i
  newReal <- real * other.real - imag * other.imag
  newImag <- real * other.imag + imag * other.real
  RETURN Complex(newReal, newImag)
END OPERATOR

OPERATOR Complex::==(REF other) RETURNS BOOLEAN
  RETURN (real = other.real) AND (imag = other.imag)
END OPERATOR

METHOD Complex::Display()
  IF imag >= 0 THEN
    OUTPUT real, " + ", imag, "i"
  ELSE
    OUTPUT real, " - ", ABS(imag), "i"
  ENDIF
END METHOD

PROCEDURE Main
  z0 <- Complex()                  // zero-argument constructor
  z1 <- Complex(3.0, 2.0)          // two-argument constructor
  z2 <- Complex(1.0, 4.0)
  z3 <- Complex(5.0)               // one-argument constructor: 5 + 0i
  sum  <- z1 + z2
  prod <- z1 * z2
```

```
  sum.Display()
  prod.Display()
  OUTPUT "z1 == z2 ? ", (z1 == z2)
END PROCEDURE
```

Note: comparing REAL values with = is exact. In real programs a small tolerance is often used (ABS(a - b) < epsilon).

Task 2: Trace of C1 x C2 with C1 = 3 + 2i, C2 = 1 + 4i

Here a = 3, b = 2 (from C1) and c = 1, d = 4 (from C2).

Step	Computation	Value
1	ac = real x other.real	3 x 1 = 3
2	bd = imag x other.imag	2 x 4 = 8
3	newReal = ac - bd	3 - 8 = -5
4	ad = real x other.imag	3 x 4 = 12
5	bc = imag x other.real	2 x 1 = 2
6	newImag = ad + bc	12 + 2 = 14
7	RETURN Complex(-5, 14); Display()	-5 + 14i

Check by algebra: (3 + 2i)(1 + 4i) = 3 + 12i + 2i + 8i^2 = 3 + 14i - 8 = -5 + 14i. The sum C1 + C2 = (3 + 1) + (2 + 4)i = 4 + 6i, and C1 == C2 is FALSE.

Question 3: Financial Currency Class with Stream I/O & Unary Negation

Task 1: Pseudo-code

Design choice: both the constructor and the input operator convert the amount to total cents, then split it with whole-number division that truncates toward zero. A negative amount therefore has negative dollars and/or negative cents, and the output operator prints the sign once.

```
CLASS Currency
  PRIVATE:
    dollars : INTEGER
    cents   : INTEGER
    METHOD Normalize(d : INTEGER, c : INTEGER)
  PUBLIC:
    CONSTRUCTOR(d : INTEGER = 0, c : INTEGER = 0)
    OPERATOR -() RETURNS Currency                       // unary negation
    FRIEND OPERATOR <<(REF outStream, REF cur : Currency) RETURNS REF STREAM
    FRIEND OPERATOR >>(REF inStream,  REF cur : Currency) RETURNS REF STREAM
ENDCLASS

METHOD Currency::Normalize(d, c)
  total   <- d * 100 + c              // everything in cents
  dollars <- total DIV 100            // truncates toward zero
  cents   <- total MOD 100            // same sign as total
END METHOD

CONSTRUCTOR Currency::Currency(d, c)
  Normalize(d, c)
END CONSTRUCTOR

OPERATOR Currency::-() RETURNS Currency
  RETURN Currency(-dollars, -cents)    // re-normalized by the constructor
END OPERATOR
```

```
FRIEND OPERATOR <<(REF outStream, REF cur) RETURNS REF STREAM
  IF cur.dollars < 0 OR cur.cents < 0 THEN
    WRITE outStream, "-"
  ENDIF
  WRITE outStream, "$", ABS(cur.dollars), "."
  IF ABS(cur.cents) < 10 THEN
    WRITE outStream, "0"               // two-digit cents
  ENDIF
  WRITE outStream, ABS(cur.cents)
  RETURN outStream                     // allows chaining
END OPERATOR

FRIEND OPERATOR >>(REF inStream, REF cur) RETURNS REF STREAM
  READ inStream INTO d
  READ inStream INTO c
  cur.Normalize(d, c)                  // direct access: friend
  RETURN inStream
END OPERATOR

PROCEDURE Main
  cur <- Currency()
  OUTPUT "Enter dollars and cents: "
  INPUT cur                            // uses FRIEND OPERATOR >>
  OUTPUT "Amount : ", cur              // uses FRIEND OPERATOR <<
  debt <- -cur                         // unary OPERATOR -
  OUTPUT "Negated: ", debt
END PROCEDURE
```

Task 2: Trace for d = 5, c = 275

Step	Operation	Values
1	INPUT cur reads d = 5, c = 275	d = 5, c = 275
2	Normalize: total = 5 x 100 + 275	total = 775 cents
3	dollars = 775 DIV 100; cents = 775 MOD 100	dollars = 7, cents = 75 (the 275 cents carried 2 dollars into 5 + 2)
4	OUTPUT cur: no negative part; cents 75 is two digits	prints $7.75
5	debt <- -cur: Currency(-7, -75)	Normalize: total = -700 + (-75) = -775
6	dollars = -775 DIV 100; cents = -775 MOD 100	dollars = -7, cents = -75
7	OUTPUT debt: negative, so prints "-" then $ 7 . 75	prints -$7.75

Other cases given in the question: Currency(2, 150) gives total 350, so $3.50. Currency(2, -150) gives total 50, so $0.50.

Question 4: Polynomial Expression with Overloaded Subscript [] & Destructor

Task 1: Pseudo-code

```
CLASS Polynomial
  PRIVATE:
    degree     : INTEGER
    coeffArray : POINTER TO REAL                 // size degree + 1
    STATIC dummy : REAL = 0.0                    // returned for an invalid index
  PUBLIC:
    CONSTRUCTOR(deg : INTEGER)
    COPY CONSTRUCTOR(REF other : Polynomial)
    DESTRUCTOR()
    OPERATOR [](index : INTEGER) RETURNS REF REAL
```

```
    OPERATOR +(REF other : Polynomial) RETURNS Polynomial
ENDCLASS

CONSTRUCTOR Polynomial::Polynomial(deg)
  degree     <- deg
  coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL
  FOR i <- 0 TO degree
    coeffArray[i] <- 0.0
  NEXT i
END CONSTRUCTOR

COPY CONSTRUCTOR Polynomial::Polynomial(REF other)    // deep copy
  degree     <- other.degree
  coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL
  FOR i <- 0 TO degree
    coeffArray[i] <- other.coeffArray[i]
  NEXT i
END COPY CONSTRUCTOR

DESTRUCTOR Polynomial::~Polynomial()
  DEALLOCATE coeffArray
  coeffArray <- NULL
END DESTRUCTOR

OPERATOR Polynomial::[](index) RETURNS REF REAL
  IF index >= 0 AND index <= degree THEN
    RETURN REF coeffArray[index]                      // lvalue: read and assign
  ELSE
    OUTPUT "Error: index ", index, " out of range"
    RETURN REF dummy                                  // keeps the reference valid
  ENDIF
END OPERATOR

OPERATOR Polynomial::+(REF other) RETURNS Polynomial
  maxDeg <- MAX(degree, other.degree)
  result <- Polynomial(maxDeg)                        // all coefficients start at 0.0
  FOR i <- 0 TO degree
    result[i] <- result[i] + coeffArray[i]
  NEXT i
  FOR i <- 0 TO other.degree
    result[i] <- result[i] + other.coeffArray[i]
  NEXT i
  RETURN result
END OPERATOR

PROCEDURE Main
  p1 <- Polynomial(2)
  p1[0] <- 5.0   p1[1] <- 2.0   p1[2] <- 3.0          // 3x^2 + 2x + 5
  p2 <- Polynomial(3)
  p2[0] <- 7.0   p2[2] <- 1.0   p2[3] <- 4.0          // 4x^3 + x^2 + 7
  p3 <- p1 + p2
  FOR i <- 3 DOWNTO 0
    OUTPUT p3[i]
  NEXT i
END PROCEDURE                                         // destructors free p3, p2, p1
```

Task 2: Trace of P1(x) + P2(x)

P1 = 3x^2 + 2x + 5 has degree 2, so coeffArray = [5, 2, 3]. P2 = 4x^3 + x^2 + 7 has degree 3, so coeffArray = [7, 0, 1, 4] (there is no x term). The result has degree MAX(2, 3) = 3 and starts as [0, 0, 0, 0].

Power of x	From P1 (loop 1)	From P2 (loop 2)	result[i] after both loops
x^0 (i = 0)	0 + 5 = 5	5 + 7 = 12	12
x^1 (i = 1)	0 + 2 = 2	2 + 0 = 2	2

Power of x	From P1 (loop 1)	From P2 (loop 2)	result[i] after both loops
x^2 (i = 2)	0 + 3 = 3	3 + 1 = 4	4
x^3 (i = 3)	not executed (P1 has no x^3 term)	0 + 4 = 4	4

Result: P3(x) = 4x^3 + 4x^2 + 2x + 12, coefficient array [12, 2, 4, 4]. Each polynomial owns its own heap array; at the end of Main the destructors free all three arrays.

Question 5: 2D Vector with Prefix/Postfix Increment & Compound Assignment

Task 1: Pseudo-code

```
CLASS Vector2D
  PRIVATE:
    x : REAL
    y : REAL
  PUBLIC:
    CONSTRUCTOR(a : REAL = 0.0, b : REAL = 0.0)
    OPERATOR ++() RETURNS REF Vector2D               // prefix
    OPERATOR ++(dummy : INTEGER) RETURNS Vector2D    // postfix
    OPERATOR +=(REF rhs : Vector2D) RETURNS REF Vector2D
    METHOD Display()
ENDCLASS

CONSTRUCTOR Vector2D::Vector2D(a, b)
  x <- a
  y <- b
END CONSTRUCTOR

OPERATOR Vector2D::++() RETURNS REF Vector2D         // prefix: ++v
  x <- x + 1.0
  y <- y + 1.0
  RETURN REF THIS                                   // modified object itself
END OPERATOR

OPERATOR Vector2D::++(dummy) RETURNS Vector2D        // postfix: v++
  snapshot <- Vector2D(x, y)                        // copy of the OLD state
  x <- x + 1.0
  y <- y + 1.0
  RETURN snapshot                                   // by value
END OPERATOR

OPERATOR Vector2D::+=(REF rhs) RETURNS REF Vector2D
  x <- x + rhs.x
  y <- y + rhs.y
  RETURN REF THIS
END OPERATOR

METHOD Vector2D::Display()
  OUTPUT "(", x, ", ", y, ")"
END METHOD

PROCEDURE Main
  v1 <- Vector2D(5.0, 8.0)
  v2 <- ++v1                  // prefix
  v3 <- v1++                  // postfix
  v1.Display()   v2.Display()   v3.Display()
  v1 += v2                    // compound assignment
  v1.Display()
END PROCEDURE
```

Task 2: Trace (v1 starts at (5.0, 8.0))

Step	Statement	v1	v2	v3
0	v1 <- Vector2D(5.0, 8.0)	(5, 8)	-	-
1	v2 <- ++v1: v1 incremented first to (6, 9); REF THIS is returned; v2 receives a copy of it	(6, 9)	(6, 9)	-
2	v3 <- v1++: snapshot = (6, 9); v1 becomes (7, 10); the snapshot is returned	(7, 10)	(6, 9)	(6, 9)
3	v1 += v2: (7 + 6, 10 + 9)	(13, 19)	(6, 9)	(6, 9)

Result after the two required statements: v1 = (7.0, 10.0), v2 = (6.0, 9.0), v3 = (6.0, 9.0). The prefix result v2 shows the value AFTER the increment, and the postfix result v3 shows the value from BEFORE the second increment. Both v2 and v3 are independent copies, so later changes to v1 do not affect them (the extra step 3 changes only v1).